MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 31 — Performance et Optimisation

Mesurer, profiler, optimiser — faire aller vite ce qui va lentement

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

29 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 31 — Performance et Optimisation **2**
0.1 timeit — Chronométrer précisément 2
0.2 cProfile — Profiler le code 3
0.3 Optimisations algorithmiques 5
0.4 NumPy — Vectorisation pour la performance 7
0.5 Mémoire — Mesurer et optimiser 8
0.6 Accélérer Python — Au-delà du pur Python 10
0.7 Ce qui m'a le plus surpris dans ce chapitre 11
0.8 Résumé du Chapitre 31 12

Chapitre 31 — Performance et Optimisation

À retenir

"Premature optimization is the root of all evil." — Donald Knuth. Avant d'optimiser, il faut **mesurer**. Python offre des outils précis pour identifier les goulots d'étranglement : `timeit` pour chronométrer, `cProfile` pour profiler, `memory_profiler` pour la mémoire. Une fois le problème localisé, les solutions vont de la réécriture algorithmique à NumPy, Cython, ou PyPy. Ce chapitre enseigne la méthode **mesurer → analyser → optimiser**.

timeit — Chronométrer précisément

timeit — mesurer le temps d'exécution

```python
import timeit
import time

# timeit.timeit()      mesure precise sur N repetitions
def methode_liste():
    return [x**2 for x in range(1000)]

def methode_generateur():
    return list(x**2 for x in range(1000))

def methode_map():
    return list(map(lambda x: x**2, range(1000)))

# Comparer les approches
n = 10_000
t_liste = timeit.timeit(methode_liste,      number=n)
t_gen   = timeit.timeit(methode_generateur, number=n)
t_map   = timeit.timeit(methode_map,        number=n)

print(f"List comprehension : {t_liste:.4f}s")
print(f"G n rateur         : {t_gen:.4f}s")
print(f"map()              : {t_map:.4f}s")

# timeit en une ligne      depuis une chaine
resultat = timeit.timeit(
    stmt   = "[x**2 for x in range(1000)]",
    number= 10_000
)
print(f"Temps : {resultat:.4f}s")
```

```python
# Classe Timer      plus de controle
timer = timeit.Timer(
    stmt  = "sorted(data)",
    setup = "import random; data=random.sample(range(10000)
        ,1000)"
)
temps = timer.repeat(repeat=5, number=1000)
print(f"Min  : {min(temps):.4f}s")
print(f"Moy  : {sum(temps)/len(temps):.4f}s")
print(f"Max  : {max(temps):.4f}s")

# Decorateur de chronom trage maison
def chronometrer(fn):
    import functools
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        debut = time.perf_counter()
        resultat = fn(*args, **kwargs)
        duree = time.perf_counter() - debut
        print(f"[{fn.__name__}] {duree*1000:.3f} ms")
        return resultat
    return wrapper

@chronometrer
def trier_million():
    import random
    data = random.sample(range(1_000_000), 1_000_000)
    return sorted(data)

trier_million()
```

cProfile — Profiler le code

cProfile — identifier les goulots d'étranglement

```python
import cProfile
import pstats
import io

# Fonction a profiler
def traitement_lourd():
    import random
    # Simulation d'un traitement complexe
    data  = [random.random() for _ in range(100_000)]
    tries = sorted(data)
    total = sum(tries)
    moy   = total / len(tries)
    ecart = (sum((x-moy)**2 for x in tries)
```

```
                / len(tries)) ** 0.5
    return moy, ecart

# Profiling simple
cProfile.run("traitement_lourd()")

# Profiling avec analyse des resultats
def profiler(fn, *args, **kwargs):
    """Profiler une fonction et afficher les stats."""
    pr = cProfile.Profile()
    pr.enable()
    resultat = fn(*args, **kwargs)
    pr.disable()

    # Trier par temps cumule
    flux = io.StringIO()
    stats = pstats.Stats(pr, stream=flux)
    stats.sort_stats("cumulative")
    stats.print_stats(15)   # top 15 fonctions

    print(flux.getvalue())
    return resultat

profiler(traitement_lourd)

# Profiler depuis la ligne de commande :
# python -m cProfile -s cumulative mon_script.py
# python -m cProfile -o output.prof mon_script.py
# snakeviz output.prof   # visualisation graphique

# line_profiler     profiler ligne par ligne
# pip install line_profiler
# D corer avec @profile, puis :
# kernprof -l -v mon_script.py

# Exemple conceptuel
# @profile   # d corateur line_profiler
def fonction_a_optimiser():
    result = []
    for i in range(10_000):            # ligne lente ?
        result.append(i ** 2)           # ligne lente ?
    return sum(result)                  # ligne lente ?
```

Optimisations algorithmiques

Choisir la bonne structure de données

```python
import timeit
import random

# RECHERCHE      liste vs set vs dict
donnees_liste = list(range(100_000))
donnees_set   = set(range(100_000))
donnees_dict  = {i: True for i in range(100_000)}
cible         = 99_999

t_liste = timeit.timeit(
    lambda: cible in donnees_liste, number=10_000)
t_set   = timeit.timeit(
    lambda: cible in donnees_set,   number=10_000)
t_dict  = timeit.timeit(
    lambda: cible in donnees_dict,  number=10_000)

print(f"Liste   : {t_liste:.4f}s  O(n)")
print(f"Set     : {t_set:.6f}s  O(1)     {t_liste/t_set:.0f}x plus rapide")
print(f"Dict    : {t_dict:.6f}s  O(1)")

# CONCATENATION      join vs += vs io.StringIO
def concat_plus(n):
    s = ""
    for i in range(n):
        s += str(i)    # cree une nouvelle chaine    chaque fois !
    return s

def concat_join(n):
    return "".join(str(i) for i in range(n))    # une seule allocation

n = 10_000
t_plus = timeit.timeit(lambda: concat_plus(n), number=100)
t_join = timeit.timeit(lambda: concat_join(n), number=100)
print(f"\nConcat += : {t_plus:.3f}s")
print(f"Join      : {t_join:.3f}s      {t_plus/t_join:.1f}x plus rapide")

# ACCES      attribut vs variable locale
class Config:
    valeur = 42

cfg = Config()
```

```python
def acces_attribut():
    total = 0
    for _ in range(100_000):
        total += cfg.valeur    # lookup a chaque fois
    return total

def acces_local():
    v = cfg.valeur             # une seule fois
    total = 0
    for _ in range(100_000):
        total += v             # variable locale     plus rapide
    return total

t_attr  = timeit.timeit(acces_attribut, number=100)
t_local = timeit.timeit(acces_local,    number=100)
print(f"\nAttribut : {t_attr:.3f}s")
print(f"Local    : {t_local:.3f}s      {t_attr/t_local:.1f}x
   plus rapide")
```

Optimisations Python idiomatiques

```python
import timeit

# 1. List comprehension vs boucle for
def boucle_for(n):
    result = []
    for i in range(n):
        if i % 2 == 0:
            result.append(i**2)
    return result

def comprehension(n):
    return [i**2 for i in range(n) if i % 2 == 0]

n = 10_000
t_for  = timeit.timeit(lambda: boucle_for(n),    number=1000)
t_comp = timeit.timeit(lambda: comprehension(n), number=1000)
print(f"for loop      : {t_for:.3f}s")
print(f"comprehension : {t_comp:.3f}s      {t_for/t_comp:.1f}x"
   )

# 2. Deballage vs indexation
coords = [(1,2), (3,4), (5,6)] * 10_000

def indexation():
    return [(c[0]+1, c[1]+1) for c in coords]

def deballage():
    return [(x+1, y+1) for x, y in coords]
```

```python

t_idx = timeit.timeit(indexation, number=100)
t_deb = timeit.timeit(deballage,  number=100)
print(f"\nIndexation : {t_idx:.3f}s")
print(f"D ballage  : {t_deb:.3f}s")

# 3. any()/all() court-circuitent
def boucle_verif(liste):
    for x in liste:
        if x > 50:
            return True
    return False

def any_verif(liste):
    return any(x > 50 for x in liste)

data = list(range(100))
t_boucle = timeit.timeit(lambda: boucle_verif(data), number=100
    _000)
t_any    = timeit.timeit(lambda: any_verif(data),    number=100
    _000)
print(f"\nboucle : {t_boucle:.3f}s")
print(f"any()  : {t_any:.3f}s")
```

NumPy — Vectorisation pour la performance

NumPy vs Python pur — comparaison de performance

```python
import numpy as np
import timeit

n = 1_000_000

# Calcul de distance euclidienne
def distance_python(x1, y1, x2, y2):
    return [(((x1[i]-x2[i])**2 +
              (y1[i]-y2[i])**2)**0.5)
            for i in range(len(x1))]

def distance_numpy(x1, y1, x2, y2):
    return np.sqrt((x1-x2)**2 + (y1-y2)**2)

# Donnees
import random
x1_py  = [random.random() for _ in range(n)]
y1_py  = [random.random() for _ in range(n)]
x2_py  = [random.random() for _ in range(n)]
y2_py  = [random.random() for _ in range(n)]
```

```python

x1_np  = np.array(x1_py)
y1_np  = np.array(y1_py)
x2_np  = np.array(x2_py)
y2_np  = np.array(y2_py)

t_py = timeit.timeit(
    lambda: distance_python(x1_py, y1_py, x2_py, y2_py),
    number=3
)
t_np = timeit.timeit(
    lambda: distance_numpy(x1_np, y1_np, x2_np, y2_np),
    number=3
)

print(f"Python pur : {t_py:.2f}s")
print(f"NumPy      : {t_np:.4f}s      {t_py/t_np:.0f}x plus
   rapide !")

# Operations NumPy optimisees
a = np.random.rand(10_000, 10_000)

# Produit matriciel     fortement optimise
t_dot = timeit.timeit(lambda: np.dot(a[:100], a[:100].T),
                      number=100)
print(f"\nProduit matriciel 100x100 : {t_dot*10:.3f}ms")

# Op rations sur les axes
t_sum  = timeit.timeit(lambda: np.sum(a),        number=100)
t_mean = timeit.timeit(lambda: np.mean(a, axis=0), number=100)
print(f"sum()  : {t_sum*10:.3f}ms")
print(f"mean() : {t_mean*10:.3f}ms")
```

Mémoire — Mesurer et optimiser

Optimiser la consommation mémoire

```python
import sys
from sys import getsizeof

# Comparer les tailles en memoire
liste_entiers  = list(range(10_000))
tuple_entiers  = tuple(range(10_000))
array_numpy    = __import__("numpy").arange(10_000)

print(f"Liste  : {getsizeof(liste_entiers):,} octets")
print(f"Tuple  : {getsizeof(tuple_entiers):,} octets")
print(f"NumPy  : {array_numpy.nbytes:,} octets")
```

```python

# __slots__      economiser la memoire des objets
class EtudiantNormal:
    def __init__(self, prenom, note):
        self.prenom = prenom
        self.note   = note

class EtudiantSlots:
    __slots__ = ["prenom", "note"]   # pas de __dict__ !

    def __init__(self, prenom, note):
        self.prenom = prenom
        self.note   = note

e_normal = EtudiantNormal("Alice", 15.5)
e_slots  = EtudiantSlots("Alice", 15.5)

print(f"\nSans slots  : {getsizeof(e_normal) + getsizeof(
    e_normal.__dict__):,} oct")
print(f"Avec slots  : {getsizeof(e_slots):,} oct")

# Generateurs vs listes     memoire paresseuse
def carres_liste(n):
    return [x**2 for x in range(n)]

def carres_gen(n):
    return (x**2 for x in range(n))

liste = carres_liste(1_000_000)
gen   = carres_gen(1_000_000)

print(f"\nListe  : {getsizeof(liste):,} octets")
print(f"G n r. : {getsizeof(gen):,} octets")
# Gener. = 112 octets quelle que soit la taille !

# Eviter les copies inutiles     vues NumPy
import numpy as np
original = np.arange(1_000_000)
vue      = original[::2]      # vue     pas de copie !
copie    = original[::2].copy()  # copie explicite

print(f"\nOriginal : {original.nbytes:,} oct")
print(f"Vue      : {vue.nbytes:,} oct (m me m moire)")
print(f"Copie    : {copie.nbytes:,} oct (nouvelle m moire)")
print(f"Vue partag e ? {np.shares_memory(original, vue)}")
```

Accélérer Python — Au-delà du pur Python

PyPy, Cython, Numba — alternatives de performance

```python
# ============================================================
# 1. PyPy      interpr teur Python JIT
# ============================================================
# Remplacer "python" par "pypy3" dans le terminal
# pypy3 mon_script.py
#     Souvent 5-50x plus rapide sur code Python pur
#     Incompatible avec certaines extensions C (NumPy partiel)

# ============================================================
# 2. Numba      JIT pour fonctions numeriques
# ============================================================
# pip install numba
from numba import jit, njit
import numpy as np
import timeit

# @njit      compilation AOT, pas de Python fallback
@njit
def mandelbrot_numba(c, max_iter=100):
    z = 0
    for i in range(max_iter):
        if abs(z) > 2:
            return i
        z = z*z + c
    return max_iter

# Version Python pure
def mandelbrot_python(c, max_iter=100):
    z = 0
    for i in range(max_iter):
        if abs(z) > 2:
            return i
        z = z*z + c
    return max_iter

# Premier appel      compilation JIT
_ = mandelbrot_numba(complex(0.3, 0.5))

t_py  = timeit.timeit(
    lambda: mandelbrot_python(complex(0.3, 0.5)),
    number=100_000
)
t_jit = timeit.timeit(
    lambda: mandelbrot_numba(complex(0.3, 0.5)),
    number=100_000
)
```

```
print(f"Python   : {t_py:.3f}s")
print(f"Numba    : {t_jit:.3f}s      {t_py/t_jit:.0f}x plus
    rapide")

# @jit avec tableaux NumPy
@njit(parallel=True)
def somme_numpy_numba(arr):
    total = 0.0
    for x in arr:
        total += x
    return total

# ==============================================================
# 3. Bonnes pratiques generales d'optimisation
# ==============================================================
# Ordre d'efficacite (du plus simple au plus complexe) :
#
# 1. Choisir le bon algorithme      O(n log n) vs O(n  )
# 2. Choisir la bonne structure     set vs list pour membership
# 3. Eviter le travail inutile      early return, short-circuit
# 4. Vectoriser avec NumPy           viter  les boucles Python
# 5. Mettre en cache                @lru_cache, @cache
# 6. Compiler avec Numba/Cython     si NumPy insuffisant
# 7. Paralleliser avec multiprocessing  si CPU-bound
# 8. Reecrire en C extension        cas extremes
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. Mesurer avant d'optimiser — la règle d'or absolue. J'avais l'habitude de "penser" que telle partie du code était lente, et d'optimiser en aveugle. Après avoir utilisé `cProfile`, j'ai découvert que mes intuitions étaient souvent complètement fausses. Le goulot d'étranglement était toujours ailleurs. Profiler avant d'optimiser n'est pas une bonne pratique — c'est la seule façon valide de procéder.

2. La structure de données fait toute la différence. Chercher un élément dans une liste de 100 000 éléments vs un set : des centaines de fois plus lent. Ce n'est pas de la micro-optimisation — c'est de la complexité algorithmique. O(n) vs O(1). Cette différence peut transformer un programme qui tourne en plusieurs minutes en un programme qui répond en millisecondes. Choisir la bonne structure de données est la première optimisation à considérer.

3. `__slots__` réduit drastiquement la mémoire. Chaque objet Python sans `__slots__` embarque un dictionnaire `__dict__` pour stocker ses attributs. Pour un million d'objets légers, ce overhead est énorme. `__slots__` supprime ce dictionnaire et stocke les attributs directement — parfois 5x moins de mémoire. Pour des Data

Scientists qui créent des millions d'objets de données, c'est une optimisation critique.

Résumé du Chapitre 31

Ce que j'ai appris dans ce chapitre :

- **Règle d'or** : mesurer avant d'optimiser — jamais deviner.
- `timeit` — chronométrer précisément sur N répétitions.
- `cProfile` + `pstats` — identifier les fonctions lentes.
- **Structures de données** : `set`/`dict` O(1) vs `list` O(n).
- `"".join()` — concaténation efficace vs `+=`.
- Variables locales, `any()`/`all()`, list comprehensions.
- **NumPy** — vectorisation, 50-100x plus rapide que Python pur.
- `__slots__` — réduire la mémoire des objets.
- **Numba** `@njit` — JIT compilation pour le calcul numérique.

"Un programme lent qui donne le bon résultat vaut mieux qu'un programme rapide qui donne le mauvais. Optimiser sans mesurer, c'est accélérer dans la mauvaise direction."